

Lab 02 — Plan and Build the Site with /plan

Agentic AI Applications with Codex · Day 1 · Codex

Course: Agentic AI Applications with Codex (C695)

Day 1 · Topic 1 · about 30 minutes · slides 34–37

Surface: Codex

Features: Add a folder · GPT-6 Sol · /plan · the 7-step workflow

The story so far

The research is back: home cooks aged 28–45 book weekend classes on their phones, and sourdough and pastry lead demand. Grace approves the line-up and wants a site she can show investors on Friday — driven by the catalogue, so a fee change never needs a developer.

Your goal

Turn the market brief and the 20-course catalogue into a real, responsive website — planned before a single file changes.

You'll build

A running site: hero with photos, 20 course cards, filters, search, campuses

What is in this folder

- `assets/courses.json`
- `assets/brand.md`
- `assets/market-brief-sample.md`
- `assets/hero-images.md`
- `solution/` — reference files from the verified build
- `prompts.md` / `prompts.pdf` — every prompt, ready to paste
- `evidence/checklist.md` — what to capture as proof

Step by step

1. **Create the project** — Make a folder `cook-and-bake`, run `git init`, and copy `courses.json` into `data/` and `brand.md` to the root.
2. **Open it in Codex** — Create a local project, then Edit project → Add folder → `cook-and-bake`. Select GPT-6 Sol.
3. **Plan first** — Type `/plan`, then paste the prompt. Read the questions Codex asks.
4. **Answer and narrow** — Answer every question. Cut or narrow one step, then approve.
5. **Serve it** — Run `python3 -m http.server 8080` — the page loads `courses.json` with `fetch()`, which fails on `file://`.

6. **Look at it** — Open <http://localhost:8080> at desktop width, then at 375px.

The prompts

PROMPT — Codex, after /plan

Build the Cook & Bake Academy website.
Read market-brief.md and brand.md first.

MUST HAVE

- Hero: headline, one-line pitch, two buttons (Browse courses, Ask our course assistant) and a collage of 3 food photos (see hero-images.md).
- Course grid rendered from data/courses.json — never hard-code a fee. Card: photo, code, level, campus, title, summary, weeks, schedule, fee.
- Filter chips All / Bakery / Cooking + a search box.
- Campuses section and an empty FAQ section.

CONSTRAINTS

- Plain HTML, CSS, JavaScript. No framework.
- Accessible: labels, alt text, keyboard focus.

DONE WHEN

- 20 cards render; Bakery shows exactly 10.
- No horizontal scroll at 375px.

Check your work

- ☐ Plan mode showed numbered steps AND questions before any edit.
- ☐ You answered its questions instead of letting it guess.
- ☐ All 20 cards render from data/courses.json — no fee in the HTML.
- ☐ Bakery shows exactly 10 courses; Cooking shows 10.
- ☐ The hero shows a three-photo collage.
- ☐ No horizontal scroll at 375px wide.

If it goes wrong

- **Images do not load** — The photos come from images.unsplash.com. On a locked-down network, the gradient placeholder shows instead — that is expected.

Stretch

- Ask Codex to add lazy-loading and width/height on every image to stop layout shift.

Why it matters: If the page is blank, open the browser console. "Failed to fetch" means you opened the file directly instead of via <http://>.

Next

Lab 3 — Project Rules and a Sign-up Form for Every Course. Keep what you built — the next lab starts from it.

Safety

Use only this lab's synthetic data and your own accounts. Never paste a real API key, password or personal data into a prompt or a file. Every email address in the data must resolve to your own inbox. Nothing is published or sent without your explicit approval.